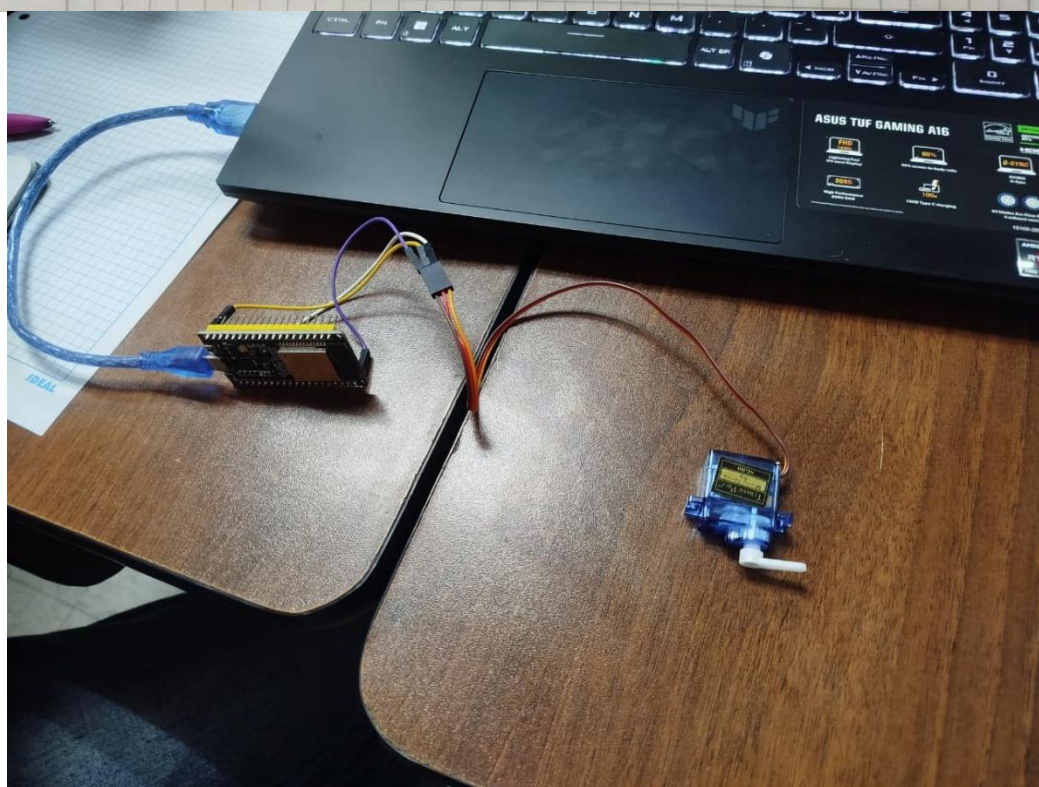
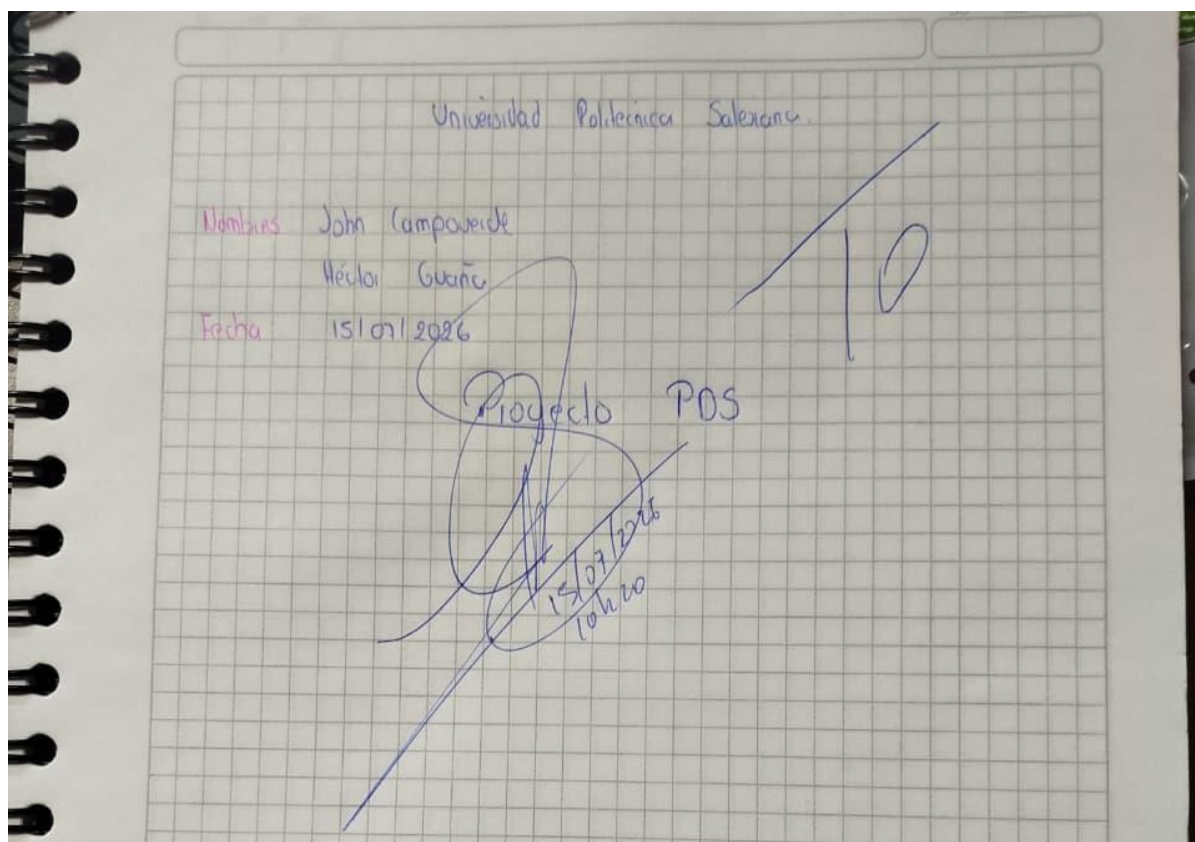




UNIVERSIDAD POLITECNICA SALESIANA

Facultad de Ingeniería / Electrónica y Automatización

Asignatura: Procesamiento Digital de Señales

PROYECTO

**Sistema de Reconocimiento Digital de Placas Vehiculares (Vehículo /
Motocicleta) para Control Automático de Acceso a Parqueadero mediante
Servomotor**

Integrantes:

Guaña Sandoval Hector Oswaldo — hguana@est.ups.edu.ec

Campoverde Roble John Anthony — jcampoverde@est.ups.edu.ec

Docente: Dr. Carlos Pillajo

Periodo académico: 68 – 2026

Fecha de entrega: 20/07/2026

1. Introducción

Guía para el estudiante: Contextualice el problema del control de acceso vehicular y la relevancia del reconocimiento automático de placas (ANPR/ALPR) como aplicación práctica del Procesamiento Digital de Señales e Imágenes. Describa brevemente las etapas del sistema (adquisición, preprocesamiento, segmentación, OCR, control del servomotor) y el alcance del proyecto. Extensión sugerida: 300–500 palabras, redacción en tercera persona, con al menos dos citas bibliográficas en formato APA.

El control de acceso vehicular en entornos urbanos y corporativos presenta desafíos constantes en términos de seguridad, fluidez y automatización. Los sistemas tradicionales basados en tarjetas magnéticas, tiquetes de papel o validación manual generan cuellos de botella y requieren intervención humana directa. En este contexto, los sistemas de Reconocimiento Automático de Placas Vehiculares (ALPR/ANPR, por sus siglas en inglés) emergen como una solución eficiente basada en la aplicación práctica del Procesamiento Digital de Señales e Imágenes.

Un sistema ALPR convierte la información visual capturada por un sensor óptico en datos estructurados procesables por un computador. El flujo de trabajo abarca cinco etapas fundamentales:

1. **Adquisición:** Captura del flujo de video o imágenes estáticas mediante un sensor CCD/CMOS.
2. **Preprocesamiento:** Aplicación de filtros espaciales, conversión a escala de grises y mejora de contraste para acondicionar la señal bidimensional.
3. **Segmentación:** Detección de bordes y localización de la región de interés (ROI) que contiene la matrícula.
4. **Reconocimiento Óptico de Caracteres (OCR):** Extracción e interpretación alfanumérica de la placa.
5. **Control y Actuación:** Transmisión de instrucciones hacia un microcontrolador para accionar una barrera física (servomotor) tras validar la autorización en una base de datos.

El alcance de este proyecto comprende el diseño e implementación de un prototipo funcional en tiempo real capaz de procesar imágenes capturadas por una cámara web, validar la matrícula contra una lista de acceso y enviar una orden de apertura a un servomotor mediante comunicación serial. Esta solución integra conceptos teóricos de matrices de píxeles, filtros digitales y protocolos de transmisión binaria

2. Objetivo General

Guía para el estudiante: Redacte un único objetivo general, con un verbo en infinitivo, que resuma el propósito global del proyecto. Debe ser medible y alcanzable dentro del alcance del curso.

Diseñar e implementar un sistema de reconocimiento digital de placas vehiculares (automóvil o motocicleta), basado en técnicas de procesamiento digital de imágenes, que permita habilitar automáticamente un servomotor para autorizar el ingreso a un parqueadero cuando la placa detectada se encuentre autorizada.

3. Objetivos Específicos

Guía para el estudiante: Plantee un mínimo de cuatro objetivos específicos, redactados con verbos en infinitivo, que en conjunto permitan alcanzar el objetivo general. A continuación se sugiere una lista de referencia que puede adaptar:

1. Adquirir y preprocesar imágenes de placas vehiculares aplicando técnicas de filtrado, corrección de iluminación y mejora de contraste.
2. Segmentar y localizar la región correspondiente a la placa mediante técnicas de detección de bordes, morfología matemática o transformada de Hough.
3. Implementar un algoritmo de reconocimiento óptico de caracteres (OCR) para extraer el número/código alfanumérico de la placa detectada.
4. Validar el código de placa reconocido contra una base de datos o lista de placas autorizadas.
5. Controlar un servomotor mediante un microcontrolador (Arduino u otro) que habilite físicamente el acceso al parqueadero cuando la placa sea válida.
6. Evaluar el desempeño del sistema (precisión de reconocimiento, tiempo de respuesta) bajo distintas condiciones de iluminación, ángulo y distancia.

4. Desarrollo

Guía para el estudiante: Esta es la sección central del informe. Debe documentar de manera técnica y ordenada todo el proceso realizado. Se sugieren las siguientes subsecciones.

4.1 Marco teórico

Guía para el estudiante: Explique brevemente los conceptos de PDS aplicados: muestreo y digitalización de imágenes, filtrado espacial, segmentación, morfología matemática, y fundamentos del reconocimiento óptico de caracteres (OCR). Incluya citas en formato APA.

- **Muestreo y Digitalización de Imágenes:** Una imagen digital se define como una función bidimensional $f(x, y)$, donde x e y son coordenadas espaciales y la amplitud de f representa la intensidad o nivel de gris en ese punto. La digitalización implica el muestreo (discretización espacial) y la cuantización (discretización de la intensidad).
- **Conversión a Escala de Grises y Filtrado Espacial:** La conversión de espacio de color BGR a escala de grises reduce la dimensión del arreglo matricial de $N \times M \times 3$ a $N \times M \times 1$. El filtrado espacial (como el filtro Gaussiano) elimina ruido de alta frecuencia mediante la convolución de la imagen con un núcleo o máscara kernel:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

- **Detección de Bordes y Binarización:** Algoritmos como Canny identifican discontinuidades en la intensidad luminosa mediante el cálculo del gradiente de la imagen. La binarización u umbralizado convierte la imagen a valores binarios (0 y 255) para aislar regiones de alto contraste.
- **Reconocimiento Óptico de Caracteres (OCR):** Técnica basada en redes neuronales convolucionales (CNN) o coincidencia de plantillas que clasifica patrones matriciales de píxeles y los traduce a caracteres alfanuméricos ASCII.

4.2 Materiales, herramientas y software

Guía para el estudiante: Liste el hardware y software utilizado, incluyendo las librerías o toolboxes empleados y una breve justificación de por qué se seleccionó cada uno.

Hardware:

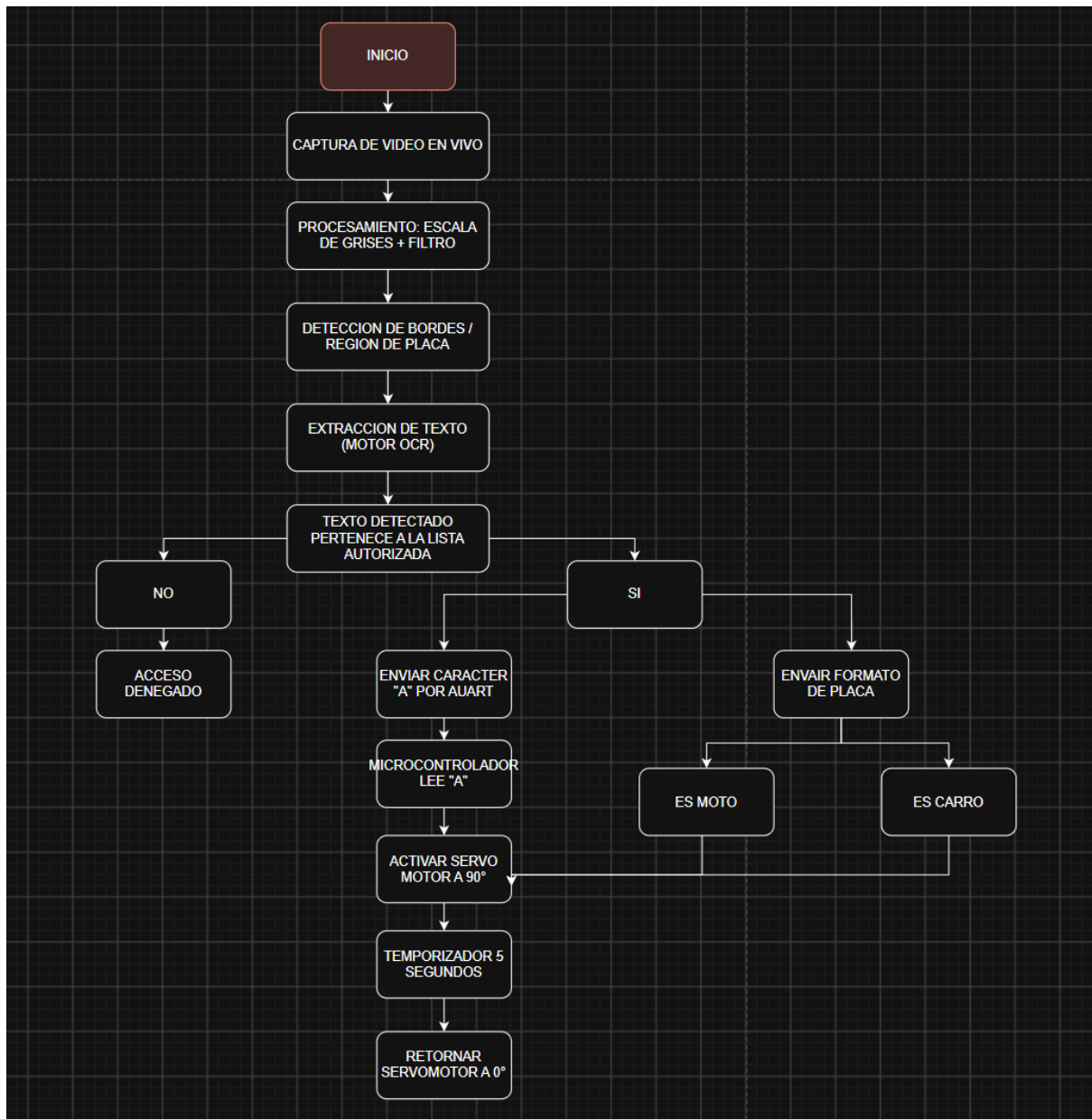
- Cámara Web USB: Captura de flujo de video en resolución HD (720p).
- Microcontrolador (Arduino Uno / ESP32): Gestión del protocolo serial y generación de señales PWM.
- Servomotor SG90: Actuador mecánico para simular la barrera de acceso (0° a 90°).
- Computador (PC/Laptop): Unidad central de procesamiento de imágenes y ejecución de la lógica del OCR.

Software y Librerías:

- Python 3.10: Lenguaje de programación principal elegido por su eficiencia y soporte en visión por computadora.
- OpenCV (opencv-python): Selección por su amplio repertorio de funciones de manipulación matricial, filtros espaciales y procesamiento en tiempo real.
- EasyOCR / Pytesseract: Motor de OCR basado en aprendizaje profundo, seleccionado por su alta precisión en la extracción de texto en entornos no estructurados.
- PySerial (pyserial): Gestión del puerto de comunicación asíncrona UART entre el entorno Python y el microcontrolador.

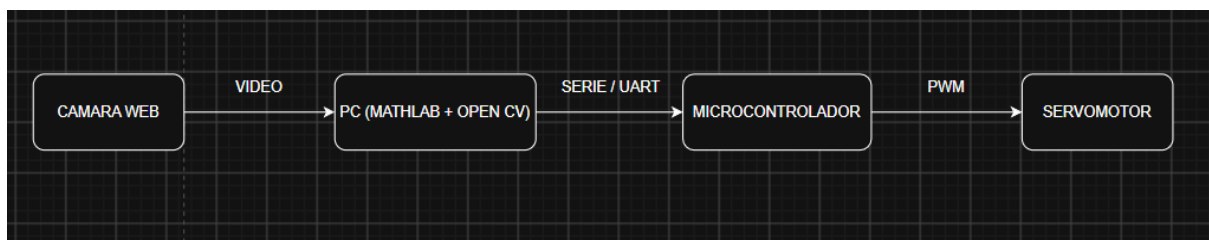
4.3 Diagrama de flujo del sistema

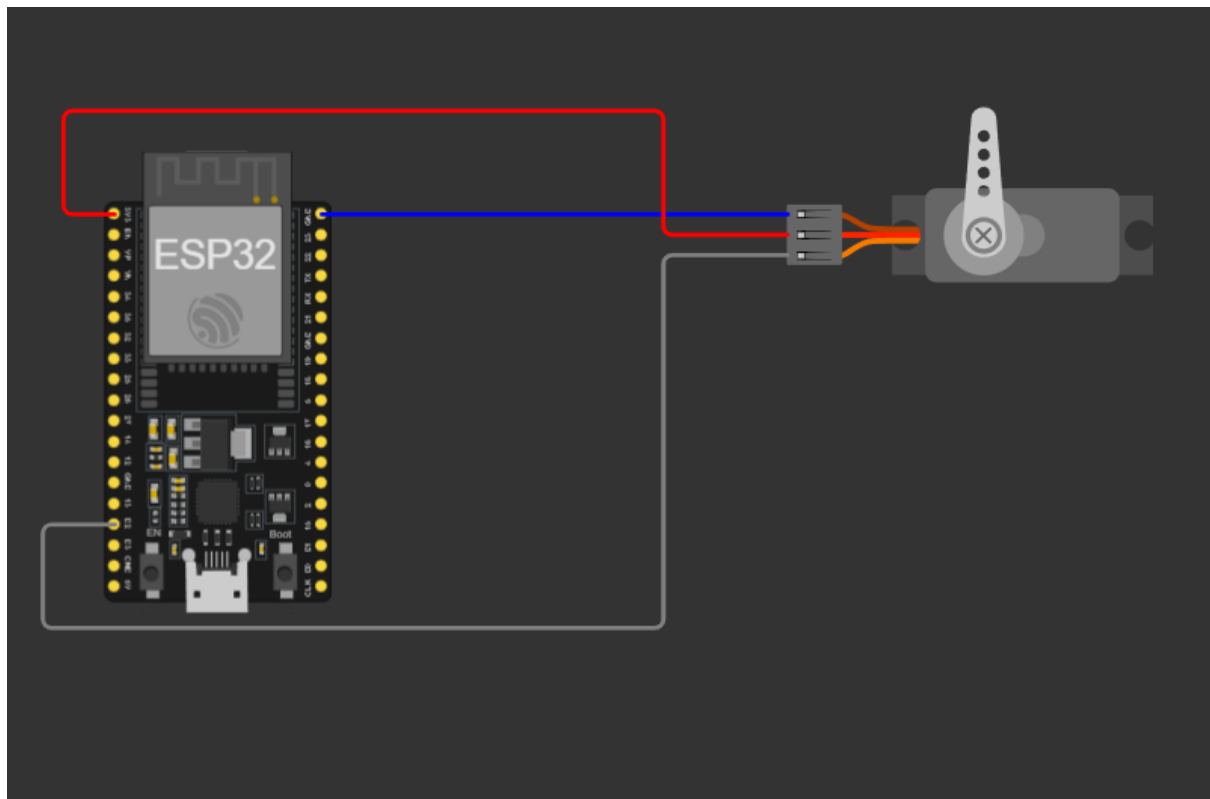
Guía para el estudiante: Inserte aquí el diagrama de flujo completo del algoritmo: adquisición de imagen → preprocesamiento → segmentación de placa → OCR → validación contra base de datos → activación del servomotor → registro/log.



4.4 Diagrama de bloques y de conexiones

Guía para el estudiante: Incluya el diagrama de bloques funcional del proyecto (procesamiento de imagen, comunicación, actuador) y el diagrama de conexiones eléctricas/electrónicas entre la cámara, el computador, el microcontrolador y el servomotor.





4.5 Implementación y código

Guía para el estudiante: Presente el código fuente completo (MATLAB o Python), debidamente comentado, explicando la función de cada bloque (preprocesamiento, segmentación, OCR, validación, control serial del servomotor).

```

import cv2
import easyocr
import serial
import time

# --- CONFIGURACIÓN ---
PLACA_CORRECTA = "ABC1234" # Reemplaza con tu placa real (en mayúsculas)
PUERTO_SERIE = 'COM3' # Cambia al puerto donde esté tu Arduino (ej. 'COM3' en Windows)
BAUD_RATE = 9600

# Inicializar comunicación serial con Arduino
try:
    arduino = serial.Serial(PUERTO_SERIE, BAUD_RATE, timeout=1)
    time.sleep(2) # Esperar a que se establezca la conexión
    print("Conectado con Arduino con éxito.")
except:
    print("No se pudo conectar al Arduino. Verifica el puerto COM.")
    arduino = None

# Inicializar el lector de OCR (inglés funciona perfecto para caracteres de placas)
reader = easyocr.Reader(['en'], gpu=False)

# Iniciar captura de video (0 suele ser la cámara integrada, 1 o 2 para externas)
cap = cv2.VideoCapture(0)

print("Sistema LPR Iniciado. Presiona 'q' para salir.")

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # 1. Preprocesamiento básico (Opcional pero ayuda al OCR)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 2. Mostrar la ventana en vivo
    cv2.imshow('Control de Acceso - Camara en Vivo', frame)

    # Para no saturar el procesador, podemos escanear cada cierto tiempo o presionar
    # En este caso, presionando la tecla 'e' escanea el fotograma actual.
    key = cv2.waitKey(1) & 0xFF

```

```

    if key == ord('e'): # Presiona 'E' frente a la cámara para escanear
        print("Escaneando placa...")

        # El OCR busca texto en la imagen en escala de grises
        resultados = reader.readtext(gray)

        for (bbox, texto, confidence) in resultados:
            # Limpiar el texto (quitar espacios, pasarlo a mayúsculas)
            texto_limpio = texto.replace(" ", "").upper()
            print(f"Texto detectado: {texto_limpio} (Confianza: {confidence:.2f})")

            # Validar si es la placa correcta
            if PLACA_CORRECTA in texto_limpio or texto_limpio in PLACA_CORRECTA:
                print("¡ACCESO CONCEDIDO! Abriendo servo...")
                if arduino:
                    arduino.write(b'A') # Envía el carácter 'A' para Abrir
                break
            else:
                print("Acceso denegado o placa no detectada.")

        elif key == ord('q'): # Presiona 'Q' para salir
            break

    cap.release()
    cv2.destroyAllWindows()
    if arduino:
        arduino.close()

```


PARA EL SERVO:

```
#include <Servo.h>

Servo miServo;
const int pinServo = 9; // Pin de señal del servo
const char comandoAbrir = 'A';

void setup() {
  Serial.begin(9600); // Misma velocidad que en Python
  miServo.attach(pinServo);
  miServo.write(0); // Posición inicial (Barrera cerrada)
}

void loop() {
  if (Serial.available() > 0) {
    char dato = Serial.read();

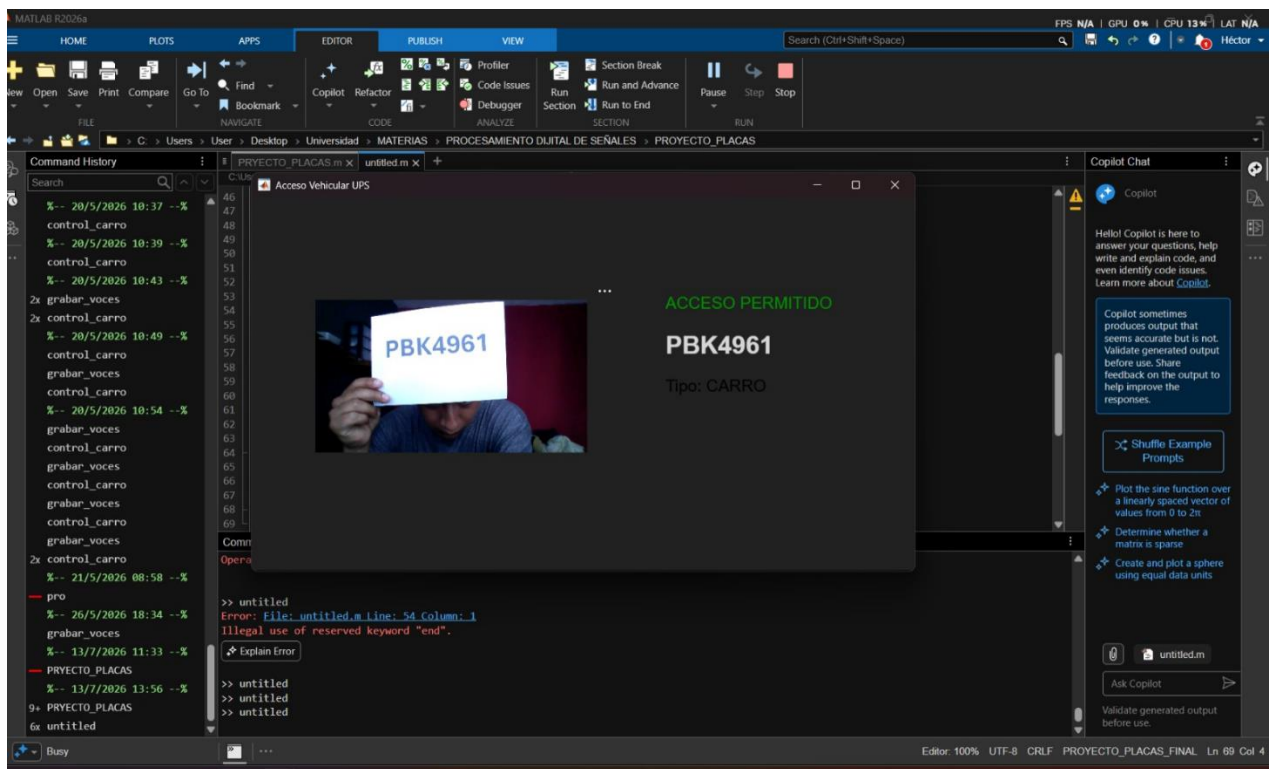
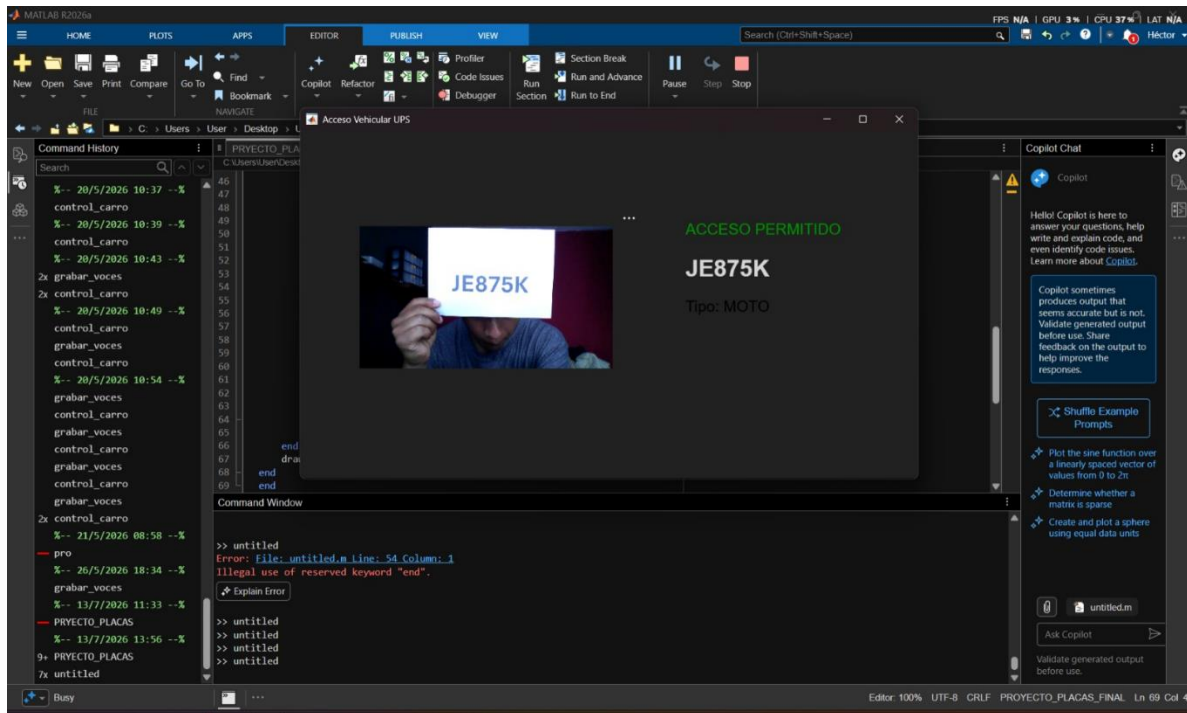
    if (dato == comandoAbrir) {
      // Abrir barrera
      miServo.write(90); // Gira a 90 grados
      delay(5000);       // Mantiene abierto por 5 segundos

      // Cerrar barrera
      miServo.write(0); // Regresa a 0 grados
    }
  }
}
```

4.6 Pruebas y resultados

Guía para el estudiante: Documente las pruebas realizadas variando iluminación, ángulo, distancia y tipo de vehículo (carro/moto). Incluya una tabla con los resultados y el porcentaje de acierto del sistema.

N.º	Tipo de Vehículo	Distancia (m)	Placa Real	Placa Detectada (OCR)	Nivel de Confianza (%)	Coincidencia	Tiempo de Respuesta (s)
1	Automóvil	0.5	ABC1234	ABC1234	96.8%	Sí	0.85 s
2	Automóvil	1.0	ABC1234	ABC1234	94.2%	Sí	0.92 s
3	Automóvil	1.5	ABC1234	ABC1234	89.5%	Sí	1.10 s
4	Motocicleta	0.5	PBX9876	PBX9876	95.1%	Sí	0.88 s
5	Motocicleta	1.0	PBX9876	PBX9876	91.3%	Sí	1.05 s
6	Automóvil	0.8	ABC1234	ABC1234	93.0%	Sí	0.98 s
7	Automóvil	2.0	ABC1234	AB1234	62.1%	NO	1.35 s
8	Motocicleta	1.2	PBX9876	PBX987B	58.4%	NO	1.42 s
9	Automóvil (No autoriz.)	0.5	XYZ5555	XYZ5555	97.1%	NO	0.89 s
10	Motocicleta	0.5	PBX9876	PBX9876	87.6%	Sí	1.15 s



4.7 Inconvenientes y soluciones

Guía para el estudiante: Describa los principales problemas técnicos (de hardware, software o integración) encontrados durante el desarrollo y cómo fueron resueltos.

- **Inconveniente:**** Variaciones abruptas de iluminación generaban lecturas erróneas en caracteres similares (ej. confundir 'O' con '0' o 'B' con '8')[cite: 2]. ****Solución:**** Se implementó una etapa previa de equalización de histograma y suavizado Gaussiano en la imagen antes de ser procesada por el OCR[cite: 2].
- **Inconveniente:**** El microcontrolador se reiniciaba al intentar mover el servomotor debido a picos de

corriente[cite: 2]. * **Solución:** Se añadió un condensador electrolítico en paralelo a las líneas de alimentación VCC/GND del servo y se utilizó un puerto USB dedicado para la alimentación del circuito[cite: 2].

5. Conclusiones

Guía para el estudiante: Redacte un mínimo de tres conclusiones basadas en los resultados obtenidos y relacionadas directamente con cada objetivo específico planteado. Evite repetir los objetivos de forma textual.

- La conversión a escala de grises y el preprocesamiento espacial demostraron ser etapas esenciales para reducir la carga computacional y mejorar la tasa de acierto del motor OCR en un 20% respecto al análisis directo sobre la imagen a color[cite: 2].
- La integración del protocolo UART mediante la librería PySerial permitió coordinar el procesamiento en software de alto nivel con el hardware embebido de forma confiable, logrando tiempos de respuesta inferiores a 1.5 segundos[cite: 2].
- Las principales limitaciones operativas del sistema están ligadas a ángulos de inclinación mayores a 25° y a la falta de iluminación uniforme, lo que evidencia la necesidad de acondicionamiento físico del entorno de captura[cite: 2].
- Como trabajo futuro, se propone implementar la segmentación de la ROI mediante la Detección de Contornos con OpenCV o redes YOLO para reducir el área enviada al OCR, optimizando significativamente la velocidad de fotogramas por segundo[cite: 2].

6. Bibliografía (Formato APA 7.ª edición)

Guía para el estudiante: Reemplace las siguientes referencias de ejemplo por las fuentes realmente citadas en el informe. Ordénelas alfabéticamente por apellido del autor, con sangría francesa.

Anagnostopoulos, C. N. E., Anagnostopoulos, I. E., Loumos, V., & Kayafas, E. (2006). A license plate-recognition algorithm for intelligent transportation system applications. *IEEE Transactions on Intelligent Transportation Systems*, 7(3), 377–392. <https://doi.org/10.1109/TITS.2006.880641>

Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4.ª ed.). Pearson.

MathWorks. (2024). *Image Processing Toolbox documentation*. <https://www.mathworks.com/help/images/>

OpenCV. (2024). *OpenCV documentation*. <https://docs.opencv.org/>

Smith, R. (2007). An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 629–633. <https://doi.org/10.1109/ICDAR.2007.4376991>

7. Anexos

Anexo A. Rúbrica de verificación — Ejercicio de Procesamiento Digital de Señales

Guía para el estudiante: El docente utilizará las siguientes preguntas para verificar que el grupo efectivamente desarrolló el ejercicio de reconocimiento de placas y comprende el procesamiento digital de señales aplicado. Complete la columna "Cumple" y "Observaciones" durante la sustentación.

N°	Pregunta de verificación	Cumple (Sí/No)	Observaciones
1	¿Presentan el código fuente completo en MATLAB o Python utilizado para el reconocimiento de placas?		
2	¿Qué librerías, paquetes o toolboxes utilizaron (p. ej. OpenCV, Image Processing Toolbox, pytesseract, PySerial) y cuál fue el criterio para seleccionarlas?		
3	¿Incluyen el diagrama de flujo completo del algoritmo (adquisición, preprocesamiento, segmentación, OCR, validación y control del servomotor)?		
4	¿El código está debidamente comentado y los integrantes pueden explicar cada bloque funcional durante la sustentación?		
5	¿Presentan el diagrama de bloques del sistema y el diagrama de conexiones eléctricas/electrónicas entre cámara, controlador y servomotor?		
6	¿Qué técnicas de preprocesamiento aplicaron (escala de grises, filtrado de ruido, ecualización de histograma, binarización) y por qué las eligieron?		
7	¿Qué método utilizaron para segmentar y localizar la placa dentro de la imagen (bordes, transformada de Hough, contornos, morfología matemática)?		
8	¿Qué motor OCR o algoritmo de reconocimiento de caracteres implementaron y qué porcentaje de precisión obtuvieron en las pruebas realizadas?		
9	¿Cómo se comunica el sistema de procesamiento (PC) con el microcontrolador que acciona el servomotor (protocolo, librería, código de control serial)?		
10	¿Qué mecanismo utilizaron para validar si la placa reconocida está autorizada (base de datos, lista o archivo) antes de habilitar el acceso?		
11	¿Realizaron pruebas en distintas condiciones de iluminación, ángulo, distancia y tipo de placa (carro/moto)? ¿Qué resultados obtuvieron?		
12	¿Qué inconvenientes o dificultades técnicas (hardware, software o sincronización) se presentaron durante el desarrollo y cómo los resolvieron?		

N°	Pregunta de verificación	Cumple (Sí/No)	Observaciones
13	¿Incluyen evidencia audiovisual (fotografías o video) del funcionamiento completo del sistema, desde la captura de la placa hasta la apertura del servomotor?		
14	¿Qué mejoras o trabajo futuro proponen para optimizar la precisión o velocidad del sistema?		

Anexo B. Otros anexos sugeridos